UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Desarrollo Web Integrado (100000ST61) · Unidad 3 · Semana 12 · Sesión 12

TALLER 12

Navegación y comunicación entre componentes en TechStore Web

Sistema de rutas de Angular · input() · output() · model() · servicios con signals · guards · lazy loading

Docente	Mag. Ing. Marco Antonio Flores Torres
Sede	UTP Lima Sur
Modalidad	En parejas, en laboratorio (puede completarse en casa)
Duración estimada	90 minutos
Versión de Angular	Angular 22 (versión estable vigente)
Estudiante 1	
Estudiante 2	

1. Datos generales y logro del taller

Curso	Desarrollo Web Integrado (100000ST61)
Unidad	Unidad 3: Integración de proyectos web (back-end y front-end)
Tema de la sesión	Sistema de rutas de Angular. Comunicación entre componentes.
Proyecto base	TechStore Web (sesión 11): catálogo de TechStore Perú SAC hecho con Angular y SASS
Producto final	Aplicación Angular con 7 pantallas navegables, carrito compartido y panel de administración protegido

Logro del taller

LOGRO

Al finalizar el taller, el estudiante implementa la navegación de una aplicación Angular mediante el sistema de rutas (parámetros de ruta y de consulta, navegación por código, ruta protegida con guard y carga diferida) y comunica componentes con input(), output(), model() y un servicio compartido con signals, aplicándolo al caso de comercio electrónico TechStore Perú.

Distribución sugerida del tiempo

Bloque	Pasos	Tiempo
Preparación del proyecto y estilos	Pasos 1 y 2	10 min
Modelo, servicios y guard	Pasos 3, 4 y 6	15 min
Componentes y rutas	Pasos 5 y 7	15 min
Layout, catálogo y detalle	Pasos 8, 9 y 10	25 min
Carrito y páginas restantes	Pasos 11 y 12	15 min
Pruebas y entrega	Paso 13 y sección 8	10 min

2. Escenario

TechStore Perú SAC es una tienda de tecnología de Lima. En las unidades 1 y 2 su equipo construyó la API REST con Spring Boot (paquete `pe.utp.techstore`), con JPA, transacciones y seguridad. En la sesión 11 iniciaste el front-end en Angular: un catálogo con tarjetas de producto y estilos SASS, todo en una sola pantalla.

Hoy la gerencia pide convertir ese catálogo en una **aplicación de varias pantallas**: cada producto debe tener su propia dirección para compartirla por WhatsApp, el cliente debe poder filtrar por categoría desde la URL, el carrito debe verse en toda la aplicación y el panel de administración solo debe abrirse después de iniciar sesión.

NOTA

En este taller los productos están dentro de un servicio de Angular (datos de prueba). En la sesión 14 reemplazarás ese servicio por llamadas a tu API Spring Boot, sin cambiar las pantallas. Por eso es importante separar bien los datos (servicios) de la vista (componentes).

3. Requisitos previos

Herramienta	Versión	Cómo verificar
Node.js	22.22.3 o superior, o 24.15 o superior (se recomienda la LTS 24)	`node -v`
npm	Viene con Node.js	`npm -v`
Angular CLI	22.x	`ng version`
Editor	Visual Studio Code con la extensión Angular Language Service	Abrir VS Code
Navegador	Chrome o Edge actualizado (con herramientas de desarrollador)	F12 abre las DevTools

CUIDADO

El Angular CLI 22 se niega a ejecutarse con versiones antiguas de Node.js y muestra el mensaje «The Angular CLI requires a minimum Node.js version…». Si te aparece, instala la versión LTS más reciente desde nodejs.org y vuelve a abrir la terminal.

Conceptos que debes recordar de la sesión 11

- Un componente tiene una clase TypeScript, una plantilla HTML y sus estilos SCSS.
- Los signals se leen con paréntesis (`producto()`) y se cambian con `set()` o `update()`.
- `@if`, `@for` (con `track`) y `@empty` controlan qué se muestra en la plantilla.
- SASS permite variables (`$rojo-utp`), anidamiento y módulos con `@use`.

4. ¿Qué vas a construir?

Mapa de rutas

URL	Componente	Qué muestra	Concepto que practicas
/	(redirección)	Envía a /inicio	redirectTo y pathMatch
/inicio	Inicio	Portada con accesos rápidos	routerLink y queryParams
/catalogo	Catalogo	Todos los productos	input(), output() y servicio
/catalogo?categoria=laptops	Catalogo	Productos filtrados	Parámetro de consulta
/productos/:id	ProductoDetalle	Ficha de un producto	Parámetro de ruta y model()
/carrito	Carrito	Productos agregados y total	Servicio compartido con signals

URL	Componente	Qué muestra	Concepto que practicas
/login	Login	Ingreso de demostración	Navegación por código
/admin	Admin (carga diferida)	Panel con indicadores	Guard y lazy loading
/cualquier-otra	NoEncontrado	Página 404	Ruta comodín **

Árbol de componentes y comunicación

Árbol de componentes

```
App
  ├── Header ..... lee CarritoService y AuthService
  └── <router-outlet />
      ├── Inicio
      ├── Catalogo ........ recibe ?categoria con input()
      │    └── ProductCard  [producto] (padre → hijo)  (agregar) (hijo → padre)
      ├── ProductoDetalle . recibe :id con input()
      │    └── SelectorCantidad  [(cantidad)] (doble sentido con model)
      ├── Carrito ......... lee CarritoService
      ├── Login ........... recibe ?volver con input()
      ├── Admin ........... carga diferida + authGuard
      └── NoEncontrado
```

Estructura final de carpetas

techstore-web/src

```
src/
├── scss/_variables.scss
├── styles.scss
└── app/
    ├── app.ts · app.html · app.scss
    ├── app.config.ts · app.routes.ts
    ├── models/producto.ts
    ├── services/producto-service.ts · carrito-service.ts · auth-service.ts
    ├── guards/auth-guard.ts
    ├── components/
    │   ├── header/ · product-card/ · selector-cantidad/
    └── pages/
        ├── inicio/ · catalogo/ · producto-detalle/ · carrito/
        └── login/ · admin/ · no-encontrado/
```

5. Desarrollo guiado paso a paso

Sigue los pasos en orden. Al final de cada uno encontrarás un recuadro **VERIFICA**: no avances hasta que se cumpla. Deja `ng serve` ejecutándose en una terminal; Angular recompila solo cada vez que guardas un archivo.

Paso 1. Preparar el proyecto

Opción A: si terminaste el laboratorio de la sesión 11, abre tu proyecto `techstore-web` y continúa en el paso 2. **Opción B:** si no lo tienes, crea uno nuevo:

Terminal

```
# 1. Verifica Node.js y npm
node -v
npm -v

# 2. Instala o actualiza el Angular CLI
npm install -g @angular/cli@latest

# 3. Crea el proyecto con SCSS y sin renderizado en servidor
ng new techstore-web --style=scss --ssr=false

# 4. Entra a la carpeta y ejecuta (abre el navegador)
cd techstore-web
ng serve -o
```

¿QUÉ HACE ESTE CÓDIGO?

- `--style=scss` hace que cada componente use SASS.
- `--ssr=false` crea una aplicación solo de navegador, suficiente para este curso.
- Angular ya genera `app.routes.ts` y `app.config.ts`: el Router viene listo para configurarse.
- Si el asistente pregunta por herramientas de IA, puedes elegir «None».

VERIFICA

El navegador muestra la página de bienvenida de Angular en http://localhost:4200 y la terminal no tiene errores en rojo.

Paso 2. Variables SASS y estilos globales

Crea la carpeta `src/scss` y dentro el archivo `_variables.scss` (el guion bajo indica que es un parcial de SASS). Luego reemplaza el contenido de `src/styles.scss`.

src/scss/_variables.scss

```
// Paleta institucional y medidas reutilizables
$rojo-utp: #E30613;
$rojo-oscuro: #A8000F;
$negro: #1A1A1A;
$gris: #F4F4F4;
$gris-texto: #6E6E6E;
$verde: #1E8449;
$radio: 10px;
```

src/styles.scss

```
@use 'scss/variables' as v;

* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  color: v.$negro;
  background: #fff;
}
.contenedor { max-width: 1100px; margin: 0 auto; padding: 24px; }
button {
  background: v.$rojo-utp; color: #fff; border: none;
  border-radius: v.$radio; padding: 8px 16px; cursor: pointer;
  &:disabled { background: v.$gris-texto; cursor: not-allowed; }
  &.secundario { background: v.$negro; }
}
a { color: v.$rojo-utp; }
```

¿QUÉ HACE ESTE CÓDIGO?

- `@use 'scss/variables' as v;` importa el módulo y sus variables se usan con el prefijo `v.` (por ejemplo `v.$rojo-utp`).
- El símbolo `&` dentro de `button` significa «este mismo botón»: `&:disabled` aplica al botón deshabilitado.
- La clase `.contenedor` centra el contenido; la usaremos en `app.html`.

Paso 3. Crear el modelo de datos

Terminal

```
ng generate interface models/producto
```

Reemplaza el contenido de `src/app/models/producto.ts`:

src/app/models/producto.ts

```
export interface Producto {
  id: number;
  nombre: string;
  precio: number;
  stock: number;
  categoria: string;
}

export interface ItemCarrito {
  producto: Producto;
  cantidad: number;
}
```

¿QUÉ HACE ESTE CÓDIGO?

Una interfaz describe la forma de un objeto, como una clase entidad de Java pero sin lógica. `Producto` refleja la entidad de tu API Spring Boot e `ItemCarrito` une un producto con la cantidad elegida.

Paso 4. Crear los servicios

Los servicios guardan los datos y la lógica que comparten varios componentes. Genera los tres:

Terminal

```
ng generate service services/producto-service
ng generate service services/carrito-service
ng generate service services/auth-service
```

4.1 ProductoService: la fuente de los productos

src/app/services/producto-service.ts

```
import { Injectable, signal } from '@angular/core';
```

```
import { Producto } from '../models/producto';

@Injectable({ providedIn: 'root' })
export class ProductoService {
  // Datos de prueba: en la sesión 14 vendrán de la API Spring Boot
  private readonly _lista = signal<Producto[]>([
    { id: 1, nombre: 'Laptop Lenovo IdeaPad', precio: 2499,
      stock: 8, categoria: 'laptops' },
    { id: 2, nombre: 'Mouse Logitech M280', precio: 49.9,
      stock: 25, categoria: 'accesorios' },
    { id: 3, nombre: 'Teclado Redragon K552', precio: 159,
      stock: 0, categoria: 'accesorios' },
    { id: 4, nombre: 'Monitor Samsung 24"', precio: 599,
      stock: 4, categoria: 'monitores' },
    { id: 5, nombre: 'Laptop HP Victus', precio: 3199,
      stock: 3, categoria: 'laptops' },
    { id: 6, nombre: 'Audífonos Sony WH-CH520', precio: 229,
      stock: 12, categoria: 'accesorios' },
  ]);

  readonly lista = this._lista.asReadonly();

  buscarPorId(id: number): Producto | undefined {
    return this._lista().find(p => p.id === id);
  }
}
```

¿QUÉ HACE ESTE CÓDIGO?

- `providedIn: 'root'` crea una sola instancia para toda la aplicación, igual que un `@Service` de Spring.
- `_lista` es privado; el exterior solo recibe `lista`, una versión de solo lectura (`asReadonly()`). Así nadie modifica los productos por accidente.
- `buscarPorId` devuelve el producto o `undefined` si no existe; lo aprovecharemos para mostrar «Producto no encontrado».

4.2 CarritoService: el estado compartido

src/app/services/carrito-service.ts

```
import { Injectable, computed, signal } from '@angular/core';
import { ItemCarrito, Producto } from '../models/producto';

@Injectable({ providedIn: 'root' })
export class CarritoService {
  private readonly _items = signal<ItemCarrito[]>([]);
  readonly items = this._items.asReadonly();

  readonly cantidadTotal = computed(() =>
    this._items().reduce((suma, i) => suma + i.cantidad, 0));

  readonly total = computed(() =>
    this._items().reduce((suma, i) => suma + i.producto.precio * i.cantidad, 0));

  agregar(producto: Producto, cantidad = 1): void {
    this._items.update(items => {
      const existe = items.find(i => i.producto.id === producto.id);
      return existe
        ? items.map(i => i.producto.id === producto.id
            ? { ...i, cantidad: i.cantidad + cantidad }
            : i)
        : [...items, { producto, cantidad }];
    });
  }

  quitar(id: number): void {
    this._items.update(items => items.filter(i => i.producto.id !== id));
  }
```

```
  vaciar(): void {
    this._items.set([]);
  }
}
```

¿QUÉ HACE ESTE CÓDIGO?

- `computed` crea valores derivados: `cantidadTotal` y `total` se recalculan solos cuando cambian los ítems.
- `agregar` usa `update`: si el producto ya está en el carrito suma la cantidad; si no, lo añade al final.
- Nunca se modifica el arreglo original: se crea uno nuevo con `map`, `filter` o el operador `...`. Así Angular detecta el cambio.
- Header, Catalogo, ProductoDetalle y Carrito inyectarán esta MISMA instancia: es la comunicación entre componentes lejanos.

4.3 AuthService: sesión simulada

src/app/services/auth-service.ts

```
import { Injectable, signal } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class AuthService {
  // Sesión simulada: en la sesión 14 se reemplaza por el JWT de Spring Security
  private readonly _logueado = signal(false);
  readonly logueado = this._logueado.asReadonly();

  ingresar(): void { this._logueado.set(true); }
  salir(): void { this._logueado.set(false); }
}
```

VERIFICA

La carpeta `src/app/services` tiene los tres servicios y `ng serve` compila sin errores.

Paso 5. Generar componentes y páginas

Separamos los **componentes reutilizables** (`components/`) de las **páginas** que muestra el Router (`pages/`).

Terminal

```
ng g c components/header
ng g c components/product-card
ng g c components/selector-cantidad

ng g c pages/inicio
ng g c pages/catalogo
ng g c pages/producto-detalle
ng g c pages/carrito
ng g c pages/login
ng g c pages/admin
ng g c pages/no-encontrado
```

Comando genera…	Clase	Rol en la aplicación
components/header	Header	Menú con enlaces y contador del carrito
components/product-card	ProductCard	Tarjeta de un producto (recibe y emite datos)
components/selector-cantidad	SelectorCantidad	Botones − / + reutilizables (model)
pages/…	Inicio, Catalogo, ProductoDetalle, Carrito, Login, Admin, NoEncontrado	Pantallas asociadas a una ruta

IMPORTANTE

En Angular 22 el CLI crea archivos sin el sufijo .component: por ejemplo `catalogo.ts`, `catalogo.html` y `catalogo.scss`, con la clase `Catalogo`. Cada comando también crea un archivo `.spec.ts` de pruebas que puedes conservar.

Paso 6. Crear el guard de autenticación

```
Terminal
ng generate guard guards/auth --implements CanActivate
```

Reemplaza el contenido de `src/app/guards/auth-guard.ts`:

```
src/app/guards/auth-guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth-service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.logueado()) {
    return true; // puede entrar
  }
  // No puede entrar: va a /login recordando a dónde quería ir
  return router.createUrlTree(['/login'], {
    queryParams: { volver: state.url },
  });
};
```

¿QUÉ HACE ESTE CÓDIGO?

- Un guard es una función que el Router ejecuta ANTES de entrar a una ruta.
- Devuelve `true` para dejar pasar, o un `UrlTree` (creado con `createUrlTree`) para redirigir.
- `state.url` es la dirección que el usuario quería visitar; la enviamos como `?volver=` para regresar después del login.

CUIDADO

El guard solo protege la navegación en el navegador. La seguridad real de los datos está en el back-end con Spring Security y JWT (sesiones 8 y 9).

Paso 7. Configurar las rutas y el Router

Reemplaza el contenido de `src/app/app.routes.ts`:

```
src/app/app.routes.ts
import { Routes } from '@angular/router';
import { Inicio } from './pages/inicio/inicio';
import { Catalogo } from './pages/catalogo/catalogo';
import { ProductoDetalle } from './pages/producto-detalle/producto-detalle';
import { Carrito } from './pages/carrito/carrito';
import { Login } from './pages/login/login';
import { NoEncontrado } from './pages/no-encontrado/no-encontrado';
import { authGuard } from './guards/auth-guard';

export const routes: Routes = [
  { path: '', redirectTo: 'inicio', pathMatch: 'full' },
```

```
  { path: 'inicio', component: Inicio, title: 'TechStore | Inicio' },
  { path: 'catalogo', component: Catalogo, title: 'TechStore | Catálogo' },
  { path: 'productos/:id', component: ProductoDetalle,
    title: 'TechStore | Detalle' },
  { path: 'carrito', component: Carrito, title: 'TechStore | Carrito' },
  { path: 'login', component: Login, title: 'TechStore | Ingresar' },
  {
    path: 'admin',
    canActivate: [authGuard],
    loadComponent: () => import('./pages/admin/admin').then(m => m.Admin),
    title: 'TechStore | Administración',
  },
  { path: '**', component: NoEncontrado,
    title: 'TechStore | Página no encontrada' },
];
```

¿QUÉ HACE ESTE CÓDIGO?

- `path` no lleva barra inicial. `:id` indica una parte variable de la URL.
- `redirectTo` con `pathMatch: 'full'` redirige solo cuando la URL está vacía.
- `title` cambia el texto de la pestaña del navegador.
- `Admin` NO se importa arriba: `loadComponent` lo descarga solo cuando alguien visita /admin (carga diferida).
- `canActivate: [authGuard]` ejecuta el guard antes de entrar.
- La ruta comodín `**` va SIEMPRE al final, porque el Router usa la primera ruta que coincide.

Ahora activa la recepción de parámetros como `input()`. Reemplaza `src/app/app.config.ts`:

src/app/app.config.ts

```
import {
  ApplicationConfig, provideBrowserGlobalErrorListeners,
} from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
  ],
};
```

¿QUÉ HACE ESTE CÓDIGO?

`withComponentInputBinding()` entrega al componente los parámetros de ruta (`:id`) y de consulta (`?categoria=`) como inputs con el mismo nombre. Sin esta línea, esos inputs llegarían vacíos.

Paso 8. Layout raíz y cabecera con navegación

El componente raíz `App` dibuja la cabecera fija y el `router-outlet`, que es el lugar donde cambia la pantalla.

src/app/app.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Header } from './components/header/header';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Header],
  templateUrl: './app.html',
  styleUrl: './app.scss',
})
export class App {}
```

src/app/app.html

```
<app-header />

<main class="contenedor">
  <router-outlet />
</main>

<footer class="pie">TechStore Perú SAC · Proyecto académico UTP</footer>
```

src/app/app.scss

```
@use '../scss/variables' as v;

.pie {
  text-align: center;
  padding: 16px;
  color: v.$gris-texto;
  border-top: 1px solid v.$gris;
}
```

Ahora la cabecera:

src/app/components/header/header.ts

```
import { Component, inject } from '@angular/core';
import { Router, RouterLink, RouterLinkActive } from '@angular/router';
import { CarritoService } from '../../services/carrito-service';
import { AuthService } from '../../services/auth-service';


@Component({
  selector: 'app-header',
  imports: [RouterLink, RouterLinkActive],
  templateUrl: './header.html',
  styleUrl: './header.scss',
})
export class Header {
  protected carrito = inject(CarritoService);
  protected auth = inject(AuthService);
  private router = inject(Router);

  salir(): void {
    this.auth.salir();
    this.router.navigate(['/inicio']);
  }
}
```

src/app/components/header/header.html

```
<header class="header">
  <a routerLink="/inicio" class="logo">TechStore Perú</a>

  <nav class="menu">
    <a routerLink="/inicio" routerLinkActive="activo"
       [routerLinkActiveOptions]="{ exact: true }">Inicio</a>
    <a routerLink="/catalogo" routerLinkActive="activo">Catálogo</a>
    <a routerLink="/admin" routerLinkActive="activo">Admin</a>
    <a routerLink="/carrito" routerLinkActive="activo">
      Carrito <span class="badge">{{ carrito.cantidadTotal() }}</span>
    </a>

    @if (auth.logueado()) {
      <button (click)="salir()">Salir</button>
    } @else {
      <a routerLink="/login" routerLinkActive="activo">Ingresar</a>
    }
  </nav>
</header>
```

src/app/components/header/header.scss

```scss
@use '../../../scss/variables' as v;

.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 24px;
  background: v.$negro;
}
.logo { color: #fff; font-weight: bold; font-size: 20px; text-decoration: none; }
.menu {
  display: flex; gap: 18px; align-items: center;
  a { color: #fff; text-decoration: none; padding-bottom: 4px; }
  a.activo { color: v.$rojo-utp; border-bottom: 2px solid v.$rojo-utp; }
}
.badge {
  background: v.$rojo-utp; color: #fff;
  border-radius: 12px; padding: 2px 8px; font-size: 12px;
}
```

¿QUÉ HACE ESTE CÓDIGO?

- `routerLink` navega sin recargar la página (no uses `href`).
- `routerLinkActive="activo"` agrega la clase `activo` al enlace de la ruta actual; el SCSS la pinta de rojo.
- `{ exact: true }` evita que «Inicio» se marque cuando estás en otra ruta que empiece igual.
- `carrito.cantidadTotal()` lee el signal del servicio: el contador se actualiza solo.
- `salir()` cierra la sesión y navega por código con `router.navigate`.

VERIFICA

Ves la cabecera negra con los enlaces. Al hacer clic en Inicio, Catálogo o Carrito cambia la pantalla sin recargar, el enlace activo se pinta de rojo y el título de la pestaña cambia. Admin te envía a /login?volver=%2Fadmin.

Paso 9. Tarjeta de producto y catálogo (input y output)

9.1 ProductCard: recibe un producto y avisa al padre

src/app/components/product-card/product-card.ts

```ts
import { Component, computed, input, output } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Producto } from '../../models/producto';

@Component({
  selector: 'app-product-card',
  imports: [RouterLink, CurrencyPipe],
  templateUrl: './product-card.html',
  styleUrl: './product-card.scss',
})
export class ProductCard {
  // Padre -> hijo
  producto = input.required<Producto>();
  // Hijo -> padre
  agregar = output<Producto>();

  agotado = computed(() => this.producto().stock === 0);

  onAgregar(): void {
    this.agregar.emit(this.producto());
  }
}
```

src/app/components/product-card/product-card.html

```
<article class="card" [class.agotado]="agotado()">
  <span class="categoria">{{ producto().categoria }}</span>
  <h3>{{ producto().nombre }}</h3>
  <p class="precio">{{ producto().precio | currency:'PEN':'S/ ' }}</p>

  <div class="acciones">
    <a [routerLink]="['/productos', producto().id]">Ver detalle</a>
    <button (click)="onAgregar()" [disabled]="agotado()">
      {{ agotado() ? 'Agotado' : 'Agregar' }}
    </button>
  </div>
</article>
```

src/app/components/product-card/product-card.scss

```
@use '../../../scss/variables' as v;

.card {
  background: v.$gris;
  border-radius: v.$radio;
  padding: 16px;
  display: flex; flex-direction: column; gap: 8px;
  &.agotado { opacity: 0.6; }
}
.categoria { font-size: 12px; text-transform: uppercase; color: v.$gris-texto; }
.precio { font-size: 20px; font-weight: bold; color: v.$rojo-utp; margin: 0; }
.acciones { display: flex; justify-content: space-between; align-items: center; }
```

¿QUÉ HACE ESTE CÓDIGO?

- `producto = input.required<Producto>()`: el padre está obligado a enviar el producto (padre → hijo).
- `agregar = output<Producto>()`: evento propio; `emit()` avisa al padre qué producto se eligió (hijo → padre).
- `[routerLink]="['/productos', producto().id]"` arma la URL /productos/3 con un arreglo de segmentos.
- La tarjeta NO conoce el carrito: solo avisa. Por eso es reutilizable en otras pantallas.

9.2 Catalogo: filtra por la URL y escucha a sus hijos

src/app/pages/catalogo/catalogo.ts

```
import { Component, computed, inject, input, signal } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';
import { ProductCard } from '../../components/product-card/product-card';
import { ProductoService } from '../../services/producto-service';
import { CarritoService } from '../../services/carrito-service';
import { Producto } from '../../models/producto';

@Component({
  selector: 'app-catalogo',
  imports: [ProductCard, RouterLink, RouterLinkActive],
  templateUrl: './catalogo.html',
  styleUrl: './catalogo.scss',
})
export class Catalogo {
  private productos = inject(ProductoService);
  private carrito = inject(CarritoService);

  // Llega desde la URL: /catalogo?categoria=laptops
  categoria = input<string>();

  categorias = ['laptops', 'accesorios', 'monitores'];
  mensaje = signal('');

  filtrados = computed(() => {
    const c = this.categoria();
```

```
    const todos = this.productos.lista();
    return c ? todos.filter(p => p.categoria === c) : todos;
  });

  alAgregar(producto: Producto): void {
    this.carrito.agregar(producto);
    this.mensaje.set(producto.nombre + ' se agregó al carrito');
  }
}
```

src/app/pages/catalogo/catalogo.html

```
<h2>Catálogo</h2>

<nav class="filtros">
  <a routerLink="/catalogo" routerLinkActive="activo"
     [routerLinkActiveOptions]="{ exact: true }">Todos</a>
  @for (c of categorias; track c) {
    <a routerLink="/catalogo" [queryParams]="{ categoria: c }"
       routerLinkActive="activo"
       [routerLinkActiveOptions]="{ exact: true }">{{ c }}</a>
  }
</nav>

@if (mensaje()) {
  <p class="aviso">{{ mensaje() }}</p>
}

<section class="grilla">
  @for (p of filtrados(); track p.id) {
    <app-product-card [producto]="p" (agregar)="alAgregar($event)" />
  } @empty {
    <p>No hay productos en esta categoría.</p>
  }
</section>
```

src/app/pages/catalogo/catalogo.scss

```
@use '../../../scss/variables' as v;

.filtros {
  display: flex; gap: 12px; margin-bottom: 16px;
  a { text-transform: capitalize; text-decoration: none; padding: 6px 12px;
      border-radius: v.$radio; background: v.$gris; color: v.$negro; }
  a.activo { background: v.$rojo-utp; color: #fff; }
}
.aviso {
  background: v.$verde; color: #fff;
  padding: 8px 12px; border-radius: v.$radio;
}
.grilla {
  display: grid; gap: 16px;
  grid-template-columns: repeat(auto-fill, minmax(230px, 1fr));
}
```

¿QUÉ HACE ESTE CÓDIGO?

- `categoria = input<string>()` recibe `?categoria=laptops`. Si la URL no lo trae, vale `undefined` y se muestran todos.
- `[queryParams]="{ categoria: c }"` crea enlaces como /catalogo?categoria=monitores.
- `(agregar)="alAgregar($event)"` escucha el output de cada tarjeta; `$event` es el producto emitido.
- `alAgregar` llama al servicio: así el contador del Header cambia aunque Header no sea padre ni hijo del catálogo.
- En los filtros, `{ exact: true }` también compara los query params, por eso solo se marca el filtro elegido.

VERIFICA

En /catalogo ves 6 tarjetas. Al elegir «laptops» la URL cambia a /catalogo?categoria=laptops y solo quedan 2. Al presionar «Agregar» aparece el aviso verde y el contador del carrito sube. El teclado Redragon aparece como «Agotado».

Paso 10. Selector de cantidad y detalle del producto (model y parámetro :id)

10.1 SelectorCantidad: enlace en ambos sentidos

src/app/components/selector-cantidad/selector-cantidad.ts

```
import { Component, input, model } from '@angular/core';

@Component({
  selector: 'app-selector-cantidad',
  templateUrl: './selector-cantidad.html',
  styleUrl: './selector-cantidad.scss',
})
export class SelectorCantidad {
  // Doble sentido: el padre envía y recibe la cantidad
  cantidad = model(1);
  max = input(10);

  sumar(): void {
    if (this.cantidad() < this.max()) this.cantidad.update(c => c + 1);
  }

  restar(): void {
    if (this.cantidad() > 1) this.cantidad.update(c => c - 1);
  }
}
```

src/app/components/selector-cantidad/selector-cantidad.html

```
<div class="selector">
  <button (click)="restar()" [disabled]="cantidad() <= 1">-</button>
  <span>{{ cantidad() }}</span>
  <button (click)="sumar()" [disabled]="cantidad() >= max()">+</button>
</div>
```

src/app/components/selector-cantidad/selector-cantidad.scss

```
.selector {
  display: inline-flex; align-items: center; gap: 12px; margin: 12px 0;
  span { min-width: 24px; text-align: center; font-weight: bold; }
}
```

¿QUÉ HACE ESTE CÓDIGO?

- `cantidad = model(1)` es un input que el hijo también puede modificar con `update()`.
- El padre lo usa con `[(cantidad)]` (la «banana en caja»): envía el valor y recibe los cambios.
- `max = input(10)` limita el botón «+» al stock disponible.

10.2 ProductoDetalle: recibe el id de la URL

src/app/pages/producto-detalle/producto-detalle.ts

```
import { Component, computed, inject, input, signal } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { Router } from '@angular/router';
import { SelectorCantidad }
  from '../../components/selector-cantidad/selector-cantidad';
import { ProductoService } from '../../services/producto-service';
import { CarritoService } from '../../services/carrito-service';

@Component({
  selector: 'app-producto-detalle',
  imports: [CurrencyPipe, SelectorCantidad],
```

```
  templateUrl: './producto-detalle.html',
  styleUrl: './producto-detalle.scss',
})
export class ProductoDetalle {
  private productos = inject(ProductoService);
  private carrito = inject(CarritoService);
  private router = inject(Router);

  // Mismo nombre que el parámetro :id de la ruta
  id = input.required<string>();

  producto = computed(() => this.productos.buscarPorId(Number(this.id())));
  cantidad = signal(1);

  agregarAlCarrito(): void {
    const p = this.producto();
    if (!p) return;
    this.carrito.agregar(p, this.cantidad());
    this.router.navigate(['/carrito']);
  }

  volver(): void {
    this.router.navigate(['/catalogo']);
  }
}
```

src/app/pages/producto-detalle/producto-detalle.html

```
@if (producto(); as p) {
  <article class="detalle">
    <span class="categoria">{{ p.categoria }}</span>
    <h2>{{ p.nombre }}</h2>
    <p class="precio">{{ p.precio | currency:'PEN':'S/ ' }}</p>
    <p>Stock disponible: {{ p.stock }}</p>

    @if (p.stock > 0) {
      <app-selector-cantidad [(cantidad)]="cantidad" [max]="p.stock" />
      <button (click)="agregarAlCarrito()">
        Agregar {{ cantidad() }} al carrito
      </button>
    } @else {
      <p class="agotado">Producto agotado</p>
    }
  </article>
} @else {
  <p>Producto no encontrado.</p>
}

<button class="secundario" (click)="volver()">Volver al catálogo</button>
```

src/app/pages/producto-detalle/producto-detalle.scss

```
@use '../../../scss/variables' as v;

.detalle {
  background: v.$gris; border-radius: v.$radio;
  padding: 24px; margin-bottom: 16px;
}
.categoria { text-transform: uppercase; font-size: 12px; color: v.$gris-texto; }
.precio { font-size: 26px; font-weight: bold; color: v.$rojo-utp; }
.agotado { color: v.$rojo-oscuro; font-weight: bold; }
```

¿QUÉ HACE ESTE CÓDIGO?

- `id = input.required<string>()` se llama igual que `:id` en la ruta; el Router lo llena gracias a `withComponentInputBinding()`.
- El parámetro siempre llega como texto: `Number(this.id())` lo convierte a número.
- `computed` vuelve a buscar el producto si el id cambia.
- `@if (producto(); as p)` crea el alias `p`; el `@else` cubre ids que no existen.
- `agregarAlCarrito` y `volver` navegan por código con `router.navigate`.

VERIFICA

/productos/4 muestra el monitor. Con «+» la cantidad sube hasta 4 (su stock) y el botón dice «Agregar 2 al carrito». Al presionarlo vas a /carrito. /productos/99 muestra «Producto no encontrado».

Paso 11. Página del carrito

src/app/pages/carrito/carrito.ts

```
import { Component, inject } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { CarritoService } from '../../services/carrito-service';

@Component({
  selector: 'app-carrito',
  imports: [CurrencyPipe, RouterLink],
  templateUrl: './carrito.html',
  styleUrl: './carrito.scss',
})
export class Carrito {
  protected carrito = inject(CarritoService);
}
```

src/app/pages/carrito/carrito.html

```
<h2>Mi carrito</h2>

@if (carrito.items().length === 0) {
  <p>Tu carrito está vacío. <a routerLink="/catalogo">Ir al catálogo</a></p>
} @else {
  <table class="tabla">
    <thead>
      <tr><th>Producto</th><th>Cantidad</th><th>Subtotal</th><th></th></tr>
    </thead>
    <tbody>
      @for (item of carrito.items(); track item.producto.id) {
        <tr>
          <td>{{ item.producto.nombre }}</td>
          <td>{{ item.cantidad }}</td>
          <td>{{ item.producto.precio * item.cantidad | currency:'PEN':'S/ ' }}</td>
          <td>
            <button (click)="carrito.quitar(item.producto.id)">Quitar</button>
          </td>
        </tr>
      }
    </tbody>
  </table>
  <p class="total">Total: {{ carrito.total() | currency:'PEN':'S/ ' }}</p>
  <button class="secundario" (click)="carrito.vaciar()">Vaciar carrito</button>
}
```

src/app/pages/carrito/carrito.scss

```
@use '../../../scss/variables' as v;

.tabla {
  width: 100%; border-collapse: collapse;
  th { background: v.$rojo-utp; color: #fff; text-align: left; padding: 8px; }
  td { padding: 8px; border-bottom: 1px solid v.$gris; }
}
.total { font-size: 22px; font-weight: bold; text-align: right; }
```

¿QUÉ HACE ESTE CÓDIGO?

- La página no guarda datos propios: todo viene de `CarritoService`.
- `quitar` y `vaciar` cambian el signal del servicio y la tabla, el total y el contador del Header se actualizan a la vez.
- El pipe `currency:'PEN':'S/ '` muestra los precios en soles.

VERIFICA

Agrega productos desde el catálogo y desde el detalle; en /carrito ves cantidades, subtotales y total. «Quitar» elimina una fila y el contador baja.

Paso 12. Páginas de inicio, login, administración y 404

12.1 Inicio

src/app/pages/inicio/inicio.ts

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-inicio',
  imports: [RouterLink],
  templateUrl: './inicio.html',
  styleUrl: './inicio.scss',
})
export class Inicio {}
```

src/app/pages/inicio/inicio.html

```
<section class="hero">
  <h1>Bienvenido a TechStore Perú</h1>
  <p>Laptops, monitores y accesorios con envío a todo Lima.</p>
  <a routerLink="/catalogo">Ver catálogo completo</a>
  <a routerLink="/catalogo" [queryParams]="{ categoria: 'laptops' }">
    Ver solo laptops</a>
  <a [routerLink]="['/productos', 1]">Producto destacado</a>
</section>
```

src/app/pages/inicio/inicio.scss

```
@use '../../../scss/variables' as v;

.hero {
  display: flex; flex-direction: column; gap: 12px;
  padding: 32px; border-radius: v.$radio; background: v.$gris;
  h1 { color: v.$rojo-utp; margin: 0; }
}
```

12.2 Login: vuelve a la página protegida

src/app/pages/login/login.ts

```
import { Component, inject, input } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '../../services/auth-service';

@Component({
  selector: 'app-login',
  templateUrl: './login.html',
  styleUrl: './login.scss',
})
export class Login {
  private auth = inject(AuthService);
  private router = inject(Router);

  // Llega desde la URL: /login?volver=/admin
  volver = input<string>();

  ingresar(): void {
    this.auth.ingresar();
    this.router.navigateByUrl(this.volver() ?? '/inicio');
  }
}
```

src/app/pages/login/login.html

```
<section class="login">
  <h2>Ingresar</h2>
  <p>Modo demostración: en la sesión 14 este botón validará usuario y
     contraseña con el JWT de la API Spring Boot.</p>

  @if (volver()) {
    <p>Necesitas iniciar sesión para ver: <strong>{{ volver() }}</strong></p>
  }
  <button (click)="ingresar()">Ingresar como administrador</button>
</section>
```

src/app/pages/login/login.scss

```
@use '../../../scss/variables' as v;

.login {
  max-width: 460px; background: v.$gris;
  padding: 24px; border-radius: v.$radio;
}
```

¿QUÉ HACE ESTE CÓDIGO?

- `volver = input<string>()` recibe `?volver=/admin` desde la URL que armó el guard.
- `navigateByUrl(this.volver() ?? '/inicio')` regresa a esa dirección; si no existe, va a inicio.
- No pongas un valor por defecto dentro de `input('/inicio')`: cuando el parámetro falta, el Router asigna `undefined` y reemplaza ese valor. Por eso usamos `??`.

12.3 Admin: carga diferida y protegida

src/app/pages/admin/admin.ts

```
import { Component, computed, inject } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { ProductoService } from '../../services/producto-service';

@Component({
  selector: 'app-admin',
  imports: [CurrencyPipe],
  templateUrl: './admin.html',
  styleUrl: './admin.scss',
})
export class Admin {
  private productos = inject(ProductoService);

  lista = this.productos.lista;
  bajoStock = computed(() => this.lista().filter(p => p.stock < 5));
  valorInventario = computed(() =>
    this.lista().reduce((suma, p) => suma + p.precio * p.stock, 0));
}
```

src/app/pages/admin/admin.html

```
<h2>Panel de administración</h2>
<p>Esta pantalla se cargó de forma diferida y está protegida por authGuard.</p>

<div class="kpis">
  <div class="kpi"><span>Productos</span><strong>{{ lista().length }}</strong></div>
  <div class="kpi"><span>Valor del inventario</span>
    <strong>{{ valorInventario() | currency:'PEN':'S/ ' }}</strong></div>
</div>

<h3>Productos con stock bajo (menos de 5)</h3>
<ul>
  @for (p of bajoStock(); track p.id) {
    <li>{{ p.nombre }}: {{ p.stock }} unidades</li>
  }
</ul>
```

src/app/pages/admin/admin.scss

```
@use '../../../scss/variables' as v;

.kpis { display: flex; gap: 16px; }
.kpi {
  background: v.$gris; border-radius: v.$radio; padding: 16px; min-width: 200px;
  display: flex; flex-direction: column;
  strong { font-size: 24px; color: v.$rojo-utp; }
}
```

12.4 NoEncontrado: página 404

src/app/pages/no-encontrado/no-encontrado.ts

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-no-encontrado',
  imports: [RouterLink],
  templateUrl: './no-encontrado.html',
  styleUrl: './no-encontrado.scss',
})
export class NoEncontrado {}
```

src/app/pages/no-encontrado/no-encontrado.html

```
<section class="error">
  <h1>404</h1>
  <p>La página que buscas no existe.</p>
  <a routerLink="/inicio">Volver al inicio</a>
</section>
```

```
src/app/pages/no-encontrado/no-encontrado.scss
@use '../../../scss/variables' as v;

.error {
  text-align: center;
  h1 { font-size: 72px; color: v.$rojo-utp; margin: 0; }
}
```

Paso 13. Compilar y comprobar la carga diferida

```
Terminal
# Detén ng serve con Ctrl + C y compila para producción
ng build
```

Busca en el resultado la sección **Lazy chunk files**. Debe aparecer un archivo llamado `admin`: es el código del panel que solo se descarga al visitar /admin.

```
Resultado esperado (los nombres de archivo varían)
Initial chunk files | Names  |  Raw size
main-XXXX.js        | main   | 107 kB
...
Lazy chunk files    | Names  |  Raw size
chunk-XXXX.js       | admin  |   1.5 kB
```

VERIFICA

`ng build` termina sin errores y muestra el chunk diferido «admin». También puedes verlo en Chrome: F12 → pestaña Network → visita /admin y observa el nuevo archivo .js.

6. Pruebas de funcionamiento

Ejecuta `ng serve` y marca cada prueba. Las pruebas 1 a 9 deben hacerse navegando con los enlaces (sin recargar), para comprobar que el carrito se conserva.

N.°	Acción o URL	Resultado esperado	✓
1	Abrir http://localhost:4200/	Redirige a /inicio y la pestaña dice «TechStore | Inicio»	
2	Clic en «Catálogo»	Se ven 6 productos; «Catálogo» se resalta en rojo	
3	Clic en el filtro «laptops»	URL /catalogo?categoria=laptops y solo 2 productos	
4	Clic en «Agregar» de una laptop	Aviso verde y el contador del carrito pasa a 1	
5	Clic en «Ver detalle» del monitor	URL /productos/4 con nombre, precio y stock	
6	Pulsar «+» varias veces	La cantidad no supera el stock (4)	
7	«Agregar N al carrito»	Navega a /carrito y el contador suma N	
8	Clic en «Quitar» en el carrito	La fila desaparece y el total se recalcula	
9	Botón Atrás del navegador	Vuelve a la pantalla anterior sin recargar	
10	Clic en «Admin» sin sesión	Redirige a /login?volver=%2Fadmin	
11	«Ingresar como administrador»	Regresa a /admin y muestra el panel	
12	Escribir /productos/99	Muestra «Producto no encontrado»	
13	Escribir /ofertas	Muestra la página 404	

N.°	Acción o URL	Resultado esperado	✓
14	Clic en «Salir»	Vuelve a /inicio; /admin vuelve a pedir login	

7. Resultado esperado



Figura 1. Catálogo filtrado con /catalogo?categoria=laptops: el filtro y el menú activos se resaltan en rojo.



Figura 2. Detalle en /productos/4 con el selector de cantidad (model).

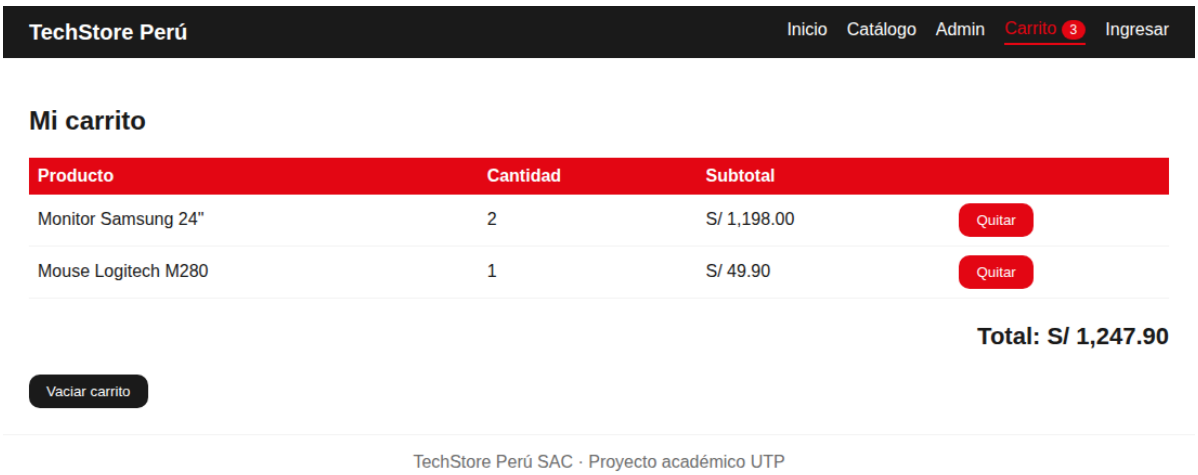


Figura 3. Carrito compartido: el contador de la cabecera coincide con la suma de cantidades.

8. Errores frecuentes y soluciones

Síntoma	Causa probable	Solución
NG8001: 'router-outlet' is not a known element	Falta RouterOutlet en `imports` de App	Agregar `RouterOutlet` al arreglo `imports`
NG8001 con 'app-product-card' o 'app-header'	El componente hijo no está en `imports` del padre	Importar la clase (por ejemplo `ProductCard`) en el padre
El enlace recarga toda la página y el carrito se vacía	Se usó `href` en vez de `routerLink`	Usar `routerLink` e importar `RouterLink`
Todas las URL muestran la página 404	La ruta `**` no está al final	Mover el comodín a la última posición
El detalle siempre dice «Producto no encontrado»	Falta `withComponentInputBinding()` o el input no se llama `id`	Revisar `app.config.ts` y que el nombre coincida con `:id`
Error NG0950: input is required but no value is available yet	Se leyó `id()` en el constructor	Leer el input dentro de `computed` o en la plantilla
«Todos» y una categoría aparecen activos a la vez	Falta `exact: true` en los filtros	Agregar `[routerLinkActiveOptions]="{ exact: true }"`
El contador del carrito no cambia	Se modificó el arreglo sin `update` o cada componente tiene su propia lista	Usar el servicio `providedIn: 'root'` y `update()` con un arreglo nuevo
Can't find stylesheet to import	Ruta incorrecta en `@use`	Contar los niveles: desde `pages/x/` es `'../../../scss/variables'`
The Angular CLI requires a minimum Node.js version	Node.js desactualizado	Instalar la LTS más reciente de Node.js

9. Retos adicionales

Elige al menos uno. Los retos suman puntos en el criterio 5 de la rúbrica.

Reto 1 · Favoritos protegidos (nivel avanzado)

- Crea `FavoritosService` con un signal de ids, los métodos `alternar(id)` y `esFavorito(id)` y un `computed total`.
- En `ProductCard` agrega `esFavorito = input(false)` y `favorito = output<number>()`.
- Muestra «Favoritos (n)» en el Header y crea la ruta `/favoritos` con `loadComponent` y `canActivate: [authGuard]`.

Reto 2 · Cantidades editables en el carrito

- Agrega `cambiarCantidad(id, cantidad)` en `CarritoService`.
- Reutiliza `SelectorCantidad` en cada fila del carrito y escucha sus cambios con `(cantidadChange)`.

Reto 3 · Rutas hijas en el panel de administración

- Convierte `/admin` en un layout con su propio `<router-outlet />` y dos rutas hijas: `/admin/inventario` y `/admin/stock-bajo`.
- Agrupa esas rutas en `admin.routes.ts` y cárgalas con `loadChildren`.

Reto 4 · Guard por rol

- Agrega un signal `rol` (`'ADMIN'` o `'CLIENTE'`) en `AuthService`.
- Crea `rolGuard` que lea `route.data['rol']` y protege `/admin` con `data: { rol: 'ADMIN' }`, como en la intranet de notas de la sesión 8.

10. Preguntas de reflexión

Responde con tus palabras en este documento (de 3 a 5 líneas por pregunta).

1. ¿Por qué el contador del carrito del Header se actualiza si Header no es padre ni hijo de Catalogo? Explica el recorrido del dato desde el clic en «Agregar».
2. ¿Qué diferencia hay entre `/productos/4` y `/catalogo?categoria=laptops`? ¿Cuándo usarías cada tipo de parámetro en tu proyecto integrador?
3. ¿Qué pasaría si colocas la ruta `**` en la primera posición del arreglo de rutas? Justifica con la regla que usa el Router.
4. ¿Por qué ProductCard emite un evento con `output()` en lugar de inyectar directamente `CarritoService`? ¿Qué ventaja tiene para reutilizar el componente?
5. El guard impide ver /admin sin sesión. ¿Por qué eso no basta para proteger los datos de la empresa? ¿Qué parte de tu proyecto Spring Boot completa esa protección?

11. Rúbrica de evaluación (20 puntos)

Criterio	Excelente (4)	Bueno (3)	En proceso (2)	Inicio (0–1)
1. Configuración de rutas	Rutas, redirección, títulos y comodín correctos y en orden	Funcionan con un detalle menor (títulos o redirección)	Faltan rutas o el comodín está mal ubicado	El Router no funciona

Criterio	Excelente (4)	Bueno (3)	En proceso (2)	Inicio (0–1)
2. Navegación	routerLink, routerLinkActive, query params y navegación por código funcionan	Falla un tipo de navegación	Usa href o recarga la página en algunos enlaces	No hay navegación entre pantallas
3. Parámetros de ruta y consulta	Detalle por :id y filtro por ?categoria con input(); maneja id inexistente	Funcionan, pero no maneja el id inexistente	Solo funciona uno de los dos	No usa parámetros
4. Comunicación entre componentes	input, output, model y servicio con signals aplicados correctamente	Usa tres de los cuatro mecanismos	Usa dos mecanismos o duplica datos entre componentes	Los componentes no se comunican
5. Guard, carga diferida, pruebas y entrega	Guard con ?volver, chunk admin, 14 pruebas, preguntas y al menos un reto	Guard y lazy loading funcionan; faltan pruebas o preguntas	Solo uno de los dos (guard o lazy)	No implementado o no entregado

12. Entregable

1. Comprime la carpeta del proyecto **sin** `node_modules` ni `dist` (por ejemplo `Taller12_Apellido1_Apellido2.zip`).
2. Incluye este documento con la tabla de pruebas marcada, las respuestas de la sección 10 y tres capturas: catálogo filtrado, detalle de producto y carrito con productos.
3. Sube ambos archivos a la actividad «Taller 12» del aula virtual en la fecha indicada por el docente.
4. Comparte en el foro del curso una dificultad que tuviste y cómo la resolviste.

PRÓXIMAS SESIONES

Guarda este proyecto: en la sesión 13 le agregarás formularios con validación y en la sesión 14 conectarás ProductoService y AuthService con tu API Spring Boot.

13. Fuentes de consulta

- Angular. (2026). Routing overview. angular.dev/guide/routing
- Angular. (2026). Accepting data with input properties. angular.dev/guide/components/inputs
- Angular. (2026). Custom events with outputs. angular.dev/guide/components/outputs
- Angular. (2026). Signals. angular.dev/guide/signals
- Angular. (2026). withComponentInputBinding (API). angular.dev/api/router/withComponentInputBinding
- Cardador Cabello, A. L. Desarrollo de aplicaciones web distribuidas (UF1846). IC Editorial. Biblioteca UTP.

Anexo para el docente: respuestas esperadas

USO DOCENTE

Esta página es para el docente. Elimínela antes de compartir el documento con los estudiantes si no desea mostrar las respuestas.

1. Ambos componentes inyectan la misma instancia de `CarritoService` (`providedIn: 'root'`). Recorrido: clic en «Agregar» → `ProductCard` emite `agregar` → `Catalogo` ejecuta `alAgregar($event)` → el servicio hace `update` del signal `_items` → `cantidadTotal` (computed) cambia → el Header, que lee ese signal en su plantilla, se vuelve a dibujar.
2. `/productos/4` es un parámetro de ruta: obligatorio, identifica un recurso único. `?categoria=laptops` es un parámetro de consulta: opcional, sirve para filtrar, buscar, ordenar o paginar. En el proyecto: `:id` para el detalle de un pedido o alumno; query params para filtros y búsquedas.
3. Todas las URL mostrarían la página 404, porque el Router recorre las rutas en orden y usa la primera coincidencia; `**` coincide con cualquier dirección.
4. Con `output()` la tarjeta solo avisa qué ocurrió y el padre decide qué hacer. Así la misma tarjeta sirve en el catálogo, en favoritos o en una lista de ofertas con comportamientos distintos, y es más fácil de probar (bajo acoplamiento).
5. El código del front-end se ejecuta en el navegador del usuario y puede modificarse o saltarse (por ejemplo, llamando directamente a la API). La protección real la da Spring Security en el back-end: validación del JWT y autorización por roles en cada endpoint.